TESTE DE SOFTWARE

ANÁLISE DE EFICÁCIA DE TESTES COM TESTE DE MUTAÇÃO

Aluno: Pedro Henrique Maia Alves
Matrícula: 848301
Data: 13 de outubro de 2026

1. Análise inicial

Inicialmente, executei os 50 testes que já existiam no projeto. Todos passaram normalmente. Em seguida, gerei o relatório de cobertura do Jest. A cobertura de linhas foi de 98,64% e a cobertura de funções chegou a 100%, o que poderia indicar que a suíte estava bem testada. Porém, a cobertura de ramos ficou em 58,82%, mostrando que vários caminhos alternativos não eram exercitados.

Depois, configurei o StrykerJS e executei o teste de mutação. Foram gerados 213 mutantes. Desse total, 154 foram mortos pelos testes, 3 terminaram em timeout, 44 sobreviveram e 12 não possuíam cobertura. O mutation score inicial foi de 73,71%.

A diferença entre os resultados mostra que cobertura de código e qualidade dos testes não são a mesma coisa. Um teste pode executar uma função inteira, aumentando a cobertura, mas ainda utilizar entradas pouco variadas ou fazer asserções insuficientes. Nesses casos, uma alteração incorreta no código pode passar sem ser percebida.

Métrica de cobertura	Resultado inicial
Instruções	85,41%
Ramos	58,82%
Funções	100%
Linhas	98,64%

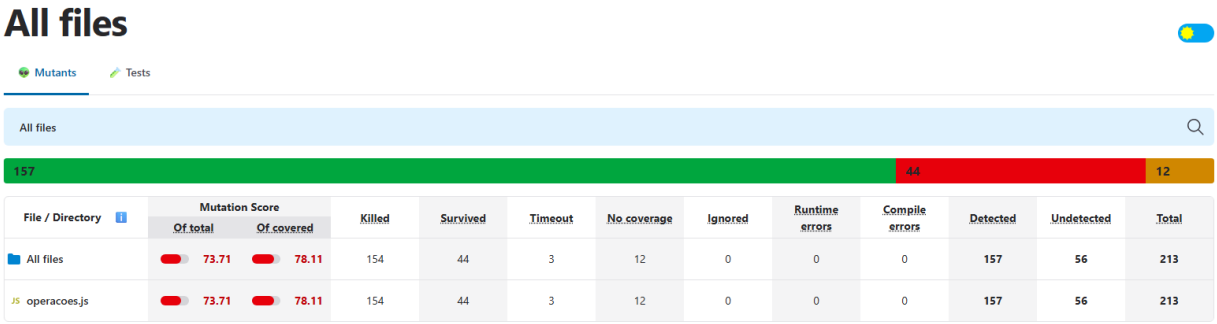


Figura 1 - Resultado inicial do teste de mutação.

2. Análise dos mutantes sobreviventes

2.1 Comparação de valores

No mutante 177, o Stryker trocou > por >= na função isMaiorQue. O teste original usava isMaiorQue(10, 5), então as duas versões retornavam verdadeiro. Percebi que faltava o caso de igualdade.

```
expect(isMaiorQue(5, 5)).toBe(false);
```

Com valores iguais, o código original retorna falso e o mutante retorna verdadeiro; por isso o novo teste o mata.



Figura 2 - Mutante 177 (EqualityOperator).

2.2 Conversão de temperatura

Nos mutantes da linha 93, operadores da fórmula foram trocados. O teste original usava somente 0 °C, valor que mascara essas trocas porque qualquer multiplicação iniciada por zero continua sendo zero.

```
expect(celsiusParaFahrenheit(100)).toBe(212);
```

Ao usar 100 °C, multiplicação e divisão passam a afetar o resultado. Assim, a alteração do mutante 155 faz o teste falhar.



Figura 3 - Mutante 155 (ArithmeticOperator).

2.3 Mediana de um array

O mutante 193 removeu o sort. O teste inicial recebia [1, 2, 3, 4, 5], que já estava ordenado e tinha tamanho ímpar. Desse modo, a remoção não mudava o resultado e o caminho par não era executado.

```
expect(medianaArray([9, 1, 5])).toBe(5);
expect(medianaArray([10, 2, 4, 8])).toBe(6);
```

O primeiro teste exige a ordenação; o segundo também verifica a média dos dois valores centrais.



Figura 4 - Mutante 193 (MethodExpression).

3. Resultados finais

Além dos três casos apresentados, adicionei testes para mensagens de erro, arrays vazios, números não primos, resultados booleanos falsos, limites do clamp e conversão inversa de temperatura. A suíte passou de 50 para 58 testes.

Depois das alterações, executei novamente o Jest e obtive 100% de cobertura em instruções, ramos, funções e linhas. Na primeira execução do Stryker após os novos testes, o mutation score chegou a 96,71%, restando sete mutantes equivalentes.

Esses sete mutantes não alteravam o resultado observável das funções. Eles estavam relacionados a condições redundantes do fatorial e do produto de arrays e aos limites do clamp. Documentei esses casos com anotações específicas do Stryker. Como as anotações são aplicadas por tipo de mutador na linha, o relatório final registrou 11 mutantes ignorados.

Na execução final, foram considerados 202 mutantes: 199 foram mortos e 3 terminaram em timeout. Não restaram mutantes sobreviventes ou sem cobertura, resultando em mutation score de 100%.

Métrica	Inicial	Final
Testes	50	58
Cobertura de ramos	58,82%	100%
Mutation score	73,71%	100%
Sobreviventes	44	0
Sem cobertura	12	0

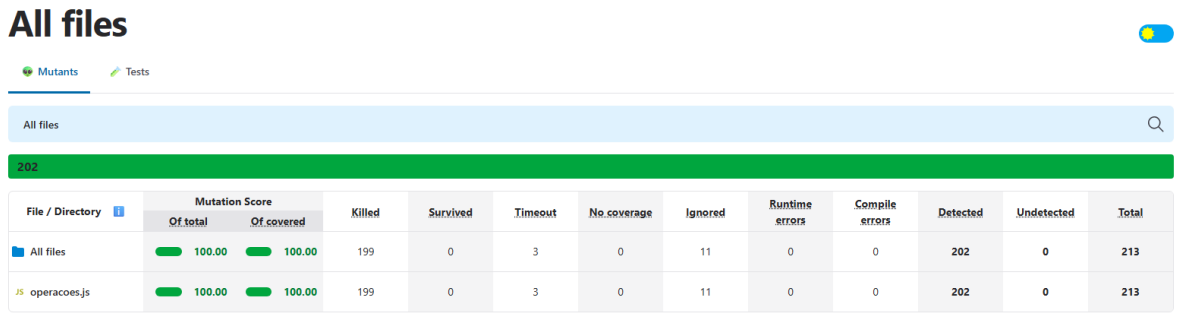


Figura 5 - Resultado final do teste de mutação.

Conclusão

A realização do trabalho mostrou que uma cobertura alta não significa necessariamente que os testes sejam capazes de detectar erros. O teste de mutação ajudou a identificar entradas pouco variadas, casos-limite ausentes e caminhos que não eram exercitados. Depois da inclusão dos novos casos, a suíte se tornou mais eficaz para detectar alterações incorretas no código.